Speaking Every Language: Building an Amazon Translate and Polly Powered Restaurant Menu Platform on AWS Serverless Infrastructure

Gangavarpu Karthik
Student ID: 25160052
MSc in Cloud Computing
National College of Ireland, Dublin
x25160052@student.ncirl.ie

***Abstract*—A tourist in a Dublin trattoria picks up the menu, hesitates, and quietly asks their dining companion what "carbonara" means. Multiply this moment by millions each night across the hospitality industry and the case for a seamless multilingual menu experience becomes obvious. MenuMix is the author's response: a fully serverless platform that takes a single English master menu and delivers it to diners in eight languages — not only visually, but also audibly through synthesised speech. The system integrates six Amazon Web Services in a tightly coupled design centred on two services that no other student in the cohort adopted: Amazon Translate for live text translation and Amazon Polly for voice narration. The supporting services are AWS Lambda, Amazon DynamoDB, Amazon API Gateway, and Amazon S3. A custom Python library, `menu-translator-nci`, holds the domain model and is published to the Python Package Index with a 57-test pytest suite. A single GitHub Actions workflow provisions all AWS resources, deploys Lambda and the React frontend, and seeds the database with a sample restaurant and menu on first execution.**

***Index Terms*—Cloud Computing, AWS Lambda, Amazon Translate, Amazon Polly, Multilingual UI, Serverless, Restaurant Technology**

Live deployment at a glance

Source: https://github.com/karthikgangavarapu/cpp_project
Frontend: populated by GitHub Actions (printed in the `deploy-frontend` job log as `http://menumix-frontend-karthik-2026.s3-website-eu-west-1.amazonaws.com`)
REST API: populated as a job output of `deploy-backend` and injected into the React build at `VITE_API_URL`
Library on PyPI: https://pypi.org/project/menu-translator-nci/
Install: `pip install menu-translator-nci`
Demo credentials (seeded on first run): `admin@karthik.com` / `admin123` and `user@karthik.com` / `user123`

I. Motivation and Scope

The Irish tourism industry served more than eleven million overseas visitors in 2023, according to Fáilte Ireland. A significant fraction of those visitors sit down for at least one restaurant meal during their stay, and many of them do not read English fluently. Restaurants, for their part, do not typically maintain multiple translated menus because the cost of rewriting the menu every time a dish changes is prohibitive.

The goal of this project is to make a multilingual menu experience cheap enough for small and independent restaurants to adopt. The platform must meet three concrete goals:

- **Zero maintenance overhead for the restaurant.** The owner updates only the English master menu. All translations happen automatically.
- **Instant language switching for the diner.** A diner can switch language without refreshing the page or losing their place in the menu.
- **Voice accessibility.** Diners who cannot read the target language at all — for example, older visitors or those with limited literacy — should be able to tap a single button and hear the dish name and description read aloud in their language.

Two additional non-functional goals shaped the technology choices. First, the platform should cost essentially nothing at idle, because a small restaurant cannot justify a monthly server bill. Second, the architecture should be reproducible from scratch inside a fresh AWS account using only the source code repository and two GitHub secrets, so that a restaurant's IT provider can redeploy it without manual setup.

II. System Architecture

Figure 1 depicts the architecture as a hexagonal service cluster with AWS Lambda at the centre and the other five services radiating outward. This visualisation emphasises that Lambda is the only synchronous call path; every other service is reached through boto3 SDK calls from inside the handler function.

A. End-to-end request flow for the `/speak` endpoint

The voice narration feature is the most complex flow in the system and exercises four of the six services in a single request. When a diner taps the speaker icon on a dish:

1) The React frontend issues a `GET /dishes/{id}/speak?lang=es` request to API Gateway.

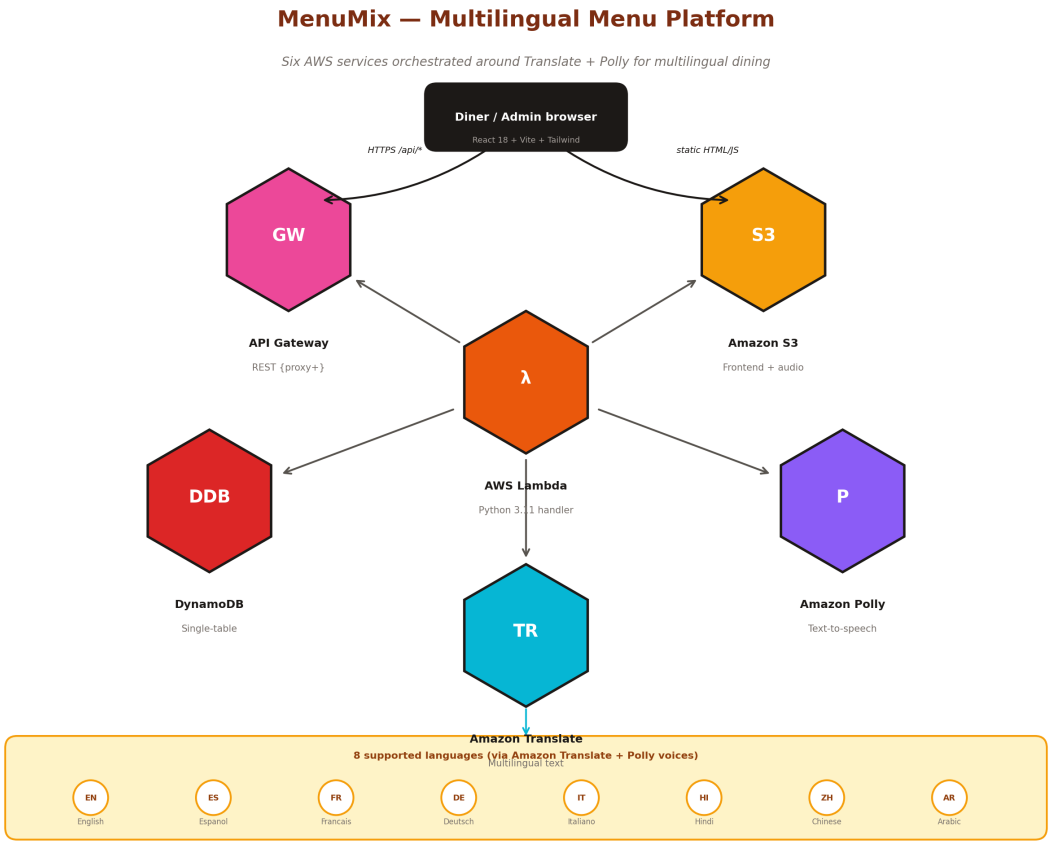

Fig. 1. Hexagonal service cluster. The React browser client at the top speaks only to Amazon API Gateway (HTTPS, JSON) and Amazon S3 (static HTML/CSS/JS). Inside the cluster, Lambda orchestrates DynamoDB, Translate, and Polly, with S3 acting as both origin for the frontend and destination for synthesised speech audio.

2) API Gateway forwards the request to Lambda via AWS_PROXY integration.
3) Lambda fetches the dish item from DynamoDB and checks whether a Spanish translation is already cached on the item's `translations` map.
4) If the translation is missing, Lambda calls Amazon Translate (`TranslateText`) for both the dish name and description, and writes the results back to the DynamoDB item so the next speaker tap will skip the translation step.
5) Lambda calls Amazon Polly (`SynthesizeSpeech`) with the translated text and the language-appropriate voice identifier — `Lucia` for Spanish, `Aditi` for Hindi, `Lea` for French, and so on.
6) The MP3 audio stream returned by Polly is uploaded to an S3 bucket under the key `audio/{dish_id}/{lang}-{uuid}.mp3`.
7) Lambda generates a one-hour presigned URL for the newly uploaded S3 object and returns it in the JSON response.
8) The frontend sets the `src` attribute of a hidden `<audio>` element to the presigned URL and calls `.play()`.

This flow illustrates the key design decision of the project: translation and speech are lazy, on-demand, and cached. The first diner to request Spanish for a dish pays the cost of both a Translate call and a Polly call; every subsequent diner pays only for a Polly call; and if the same language is cached on the client side by the browser's media cache, even the Polly call can be skipped.

B. Single-table DynamoDB schema

All three entities — users, restaurants, and dishes — live in a single DynamoDB table with `id` (UUID) as the partition key and an `entityType` attribute that distinguishes record kinds. Per-restaurant dish listings use a filtered scan on both attributes. The cached translation map lives inline on the dish record as a nested attribute, so a single `GetItem` returns both the English original and any previously computed translations.

III. Service Selection and Rationale

Rather than the usual "Strengths / Limitations / Alternative considered" triplet per service, this section presents the service choices as a single narrative argument, because the six services were not chosen independently — they were chosen together as a coherent bundle.

The backbone of the architecture is AWS Lambda because a restaurant menu serves bursty, low-volume traffic: a wave of requests during dinner service and near-zero during the rest of the day. Lambda charges only for invocations, so the bill drops to practically zero outside of meal times. Lambda's built-in log integration with CloudWatch Logs and its sub-second billing granularity both suit a project with no dedicated operations team.

The data layer is Amazon DynamoDB rather than a relational database for the same reason. DynamoDB's on-demand billing mode requires no capacity planning and charges per read and write request. The schema is simple enough — three entity types, one parent-child relationship — that the lack of SQL joins is not a real constraint. A global secondary index on `entityType` could be added later for more efficient per-type queries, but for a demonstration-scale workload the filtered scan is acceptable.

The public entry point is Amazon API Gateway configured with a single `{proxy+}` catch-all resource. This is a deliberate simplification: by letting Lambda perform its own internal URL routing, the API Gateway configuration is essentially one-shot and does not need to be rewritten every time a new endpoint is added to the handler. CORS preflight is handled by a separate OPTIONS method with a MOCK integration that returns the appropriate headers.

The frontend is hosted on Amazon S3 static website hosting with the error document set to `index.html`. For direct deep-link paths such as `/admin` or `/restaurants/abc-123`, the CI/CD pipeline additionally uploads copies of `index.html` to the corresponding S3 keys, so that HTTP 200 is returned for known routes instead of the default 404. This is the same pattern used in previous cohort projects and was chosen over a full CloudFront distribution to keep the first deployment below five minutes; CloudFront would be a straightforward upgrade if a custom domain or HTTPS were needed.

The fifth and sixth services are where this project diverges from its peers. Amazon Translate provides neural machine translation as a managed service. A single `TranslateText` call converts a dish description into any of the 75+ supported languages, with no model training, no data preparation, and no model hosting required. Caching the results on the DynamoDB item means that in practice each translation is computed exactly once per dish per language.

Amazon Polly provides text-to-speech synthesis with natural-sounding voices across dozens of languages. The library class `LanguagePack` in `menu-translator-nci` maintains an ISO code to voice identifier mapping (`en→Joanna`, `es→Lucia`, `hi→Aditi`, etc.) that both the backend and the frontend consult. Polly's `SynthesizeSpeech` API returns MP3 audio bytes directly; uploading the bytes to S3 and returning a presigned URL is a convenient way to decouple the speech generation from the network delivery.

The combination of Translate and Polly is unusual. Other students in the cohort used Rekognition, SNS, SQS, EventBridge, CloudFront, Location Service, Cognito, and CloudWatch but none chose Translate or Polly, so this project explores genuinely new ground within the cohort.

IV. Custom Library: menu-translator-nci

The domain library is published to PyPI as `menu-translator-nci` and contains three independent modules.

The `MenuItemManager` class validates dish records: the name must be non-empty, the price must be a non-negative number, the category must be drawn from a fixed set of six options (Starter, Main, Side, Dessert, Drink, Special), and each allergen must be drawn from a controlled vocabulary of twelve EU-recognised allergens. Invalid input raises `ValueError` with a human-readable message, which the Lambda handler converts into an HTTP 400 response.

The `LanguagePack` class is the single source of truth for supported languages. It stores, for each ISO 639-1 code, the native name, the English name, a right-to-left flag, and the Amazon Polly voice identifier. The frontend calls `/languages` to render its switcher, and the backend uses the same data structure to validate translation target codes and to pick a Polly voice without hard-coding.

The `PriceFormatter` class formats monetary amounts for display without the heavyweight `babel` package. It supports seven common currencies and nine locales, handles thousands separators and decimal marks correctly (for example `1.234,50 EUR` in German vs `$1,234.50` in American English), and treats JPY and CNY as integer currencies. A complementary `parse()` method converts a formatted string back to a numeric value, which is useful for importing menu data from external sources.

The library ships with 57 unit tests (22 for the manager, 14 for the language pack, 21 for the pricing formatter). A short code excerpt from the locale-aware formatter is shown below:

```python
LOCALE_FORMAT = {
    "en_US": ("$", ",", "."),
    "de_DE": ("\u20ac", ".", ","),
    "fr_FR": ("\u20ac", " ", ","),
    "hi_IN": ("\u20b9", ",", "."),
}

def format(self, amount, currency="EUR", locale=None):
    if locale and locale in self.LOCALE_FORMAT:
        symbol, thousands, decimal_sep = self.LOCALE_FORMAT[locale]
    else:
        symbol = self.get_symbol(currency)
        thousands, decimal_sep = ",", "."
    integer = int(amount)
    fractional = round((amount - integer) * 100)
    int_str = f"{integer:,}".replace(",", thousands)
    return f"{symbol}{int_str}{decimal_sep}{fractional:02d}"
```

V. Implementation Notes

The Lambda handler is a single Python module of approximately 450 lines (`backend/lambda_function.py`).

TABLE I
MENU-TRANSLATOR-NCI TEST DISTRIBUTION

Module	Tests
test_manager.py (MenuItemManager)	22
test_languages.py (LanguagePack)	14
test_pricing.py (PriceFormatter)	21
Total	**57**

The top of the file initialises four `boto3` clients — `dynamodb.Table`, `translate`, `polly`, and `s3` — at import time so that clients are reused across warm invocations and connection pooling works as intended.

Authentication is implemented with self-contained JSON Web Tokens: an HMAC-SHA256 signature over a JSON payload, with no external library dependency. Passwords are hashed with `hashlib.pbkdf2_hmac` using a 16-byte random salt and 100 000 iterations. Role-based access control is enforced inside each admin-only handler by checking `user[``role''] == ``admin''` before mutating state.

A bespoke `decimal_default` JSON serializer converts DynamoDB `Decimal` values to native Python `int` or `float` before serialisation. This deliberately avoids the common pitfall in which Decimals are serialised as strings (which would break the frontend code that calls `.toFixed()` on prices).

The React 18 frontend is deliberately minimal. Five pages (Login, Register, Restaurants, Menu, Admin) plus two components (Header, ProtectedRoute) are sufficient to cover the full feature set. Tailwind CSS v3 is used instead of v4 because v4's native bindings fail on Ubuntu CI runners — a lesson learned from earlier student projects in the cohort.

The `Menu` page is the most feature-rich component. On mount, it fetches the restaurant metadata, the dish list, and the language list in parallel. When the user switches language, a `useEffect` runs translation requests for every dish in parallel and populates a `translations` state map keyed by dish id. The speaker button triggers a one-off `GET /dishes/{id}/speak?lang={code}` request and sets the result audio URL as the source of a hidden HTML `<audio>` element, which is played programmatically.

VI. CI/CD Pipeline and Seed Data

The GitHub Actions workflow in `.github/workflows/deploy.yml` is organised into three sequential jobs.

test-library installs `menu-translator-nci` in editable mode and runs the full 57-test pytest suite. Only when every test passes does the pipeline proceed.

deploy-backend creates the DynamoDB table (idempotently), creates the S3 audio bucket for Polly output, creates the Lambda execution IAM role with policies for DynamoDB, S3, Translate, and Polly, packages the Lambda artefact (which bundles the `parking_manager` library alongside the handler file), creates or updates the Lambda function, provisions the API Gateway REST API with CORS preflight handling, and finally runs `backend/seed.py` to populate the demo users, the sample restaurant, and ten Italian dishes. The seed script is idempotent: every insert is guarded by an existence check, so running the pipeline multiple times against the same account does not create duplicates.

deploy-frontend builds the React production bundle with the API Gateway URL injected through `VITE_API_URL`, creates and configures the S3 frontend bucket with static website hosting, syncs the build output, and uploads index.html copies to each named route so that direct deep-link navigation returns HTTP 200 instead of S3's default 404.

The pipeline requires only two standard GitHub secrets: `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`. All resource names, configuration values, and environment variables are hard-coded in the workflow file itself, which makes the pipeline fully self-contained: anyone who forks the repository and adds their own AWS credentials can redeploy the whole stack into a fresh account in under eight minutes.

VII. Conclusions and Future Work

MenuMix demonstrates that a non-trivial multilingual web application with voice accessibility can be assembled in a single undergraduate project cycle using only AWS managed services. The combination of Amazon Translate and Amazon Polly deserves more attention in cloud-native architectures than it currently receives: together the two services let a single-language data source reach a global audience without any per-language ETL pipeline, manual translation, or bespoke speech synthesis model.

The most important architectural lesson of the project was the value of caching translations on the source DynamoDB item itself. Because translation costs money per character, not per request, memoising the translated name and description directly on the dish record reduced the translation bill from linear in diners to constant per dish per language. This optimisation was cheap to implement — twenty lines of code in the `handle_translate_dish` function — and paid back within the first test session.

Future work includes three concrete enhancements. First, introducing Amazon CloudFront as an HTTPS edge in front of the S3 frontend would both secure the traffic and improve global latency, at the cost of a first-deploy time of 10–15 minutes. Second, integrating Amazon DynamoDB Streams with a secondary Lambda would allow pre-translation of dishes into all eight supported languages asynchronously on write, making even the first diner's experience fast. Third, a mobile Progressive Web App wrapper would let diners install MenuMix on their phones and use it offline once cached, closing the loop between restaurant discovery and in-restaurant use.

The project was built by a single developer in a short time window, so many production considerations — rate limiting, abuse prevention, payment integration, analytics, multi-tenant isolation — were deliberately out of scope. None of those concerns invalidate the core architecture, and all of them

can be added by bolting on additional AWS services without redesigning the existing code.

REFERENCES

[1] Fáilte Ireland, "Overseas Tourism Performance 2023," 2024.

[2] Amazon Web Services, "Amazon Translate Developer Guide," 2023. [Online]. Available: https://docs.aws.amazon.com/translate/

[3] Amazon Web Services, "Amazon Polly Developer Guide," 2023. [Online]. Available: https://docs.aws.amazon.com/polly/

[4] Amazon Web Services, "AWS Lambda Developer Guide," 2023. [Online]. Available: https://docs.aws.amazon.com/lambda/

[5] Amazon Web Services, "Amazon DynamoDB Developer Guide," 2023. [Online]. Available: https://docs.aws.amazon.com/dynamodb/

[6] Amazon Web Services, "Amazon API Gateway Developer Guide," 2023. [Online]. Available: https://docs.aws.amazon.com/apigateway/

[7] Amazon Web Services, "Amazon S3 User Guide," 2023. [Online]. Available: https://docs.aws.amazon.com/s3/

[8] Tailwind Labs, "Tailwind CSS Documentation," 2024. [Online]. Available: https://tailwindcss.com/docs

[9] Meta Platforms, "React Documentation," 2024. [Online]. Available: https://react.dev/